Set Data Structure Code Along (CodeGrade)

(https://github.com/learn-co-curriculum/python-p3-dsa-set-codealong) (https://github.com/learn-co-curriculum/python-p3-dsa-set-codealong/issues/new)

Learning Goals

- Identify the use cases for a `Set`.
- Implement common methods for a `Set`.

Key Vocab

- **Sequence**: a data structure in which data is stored and accessed in a specific order.
- **Stack** is a linear data structure that follows the principle of Last In First Out (LIFO).
- **Index**: the location, represented by an integer, of an element in a sequence.
- **Iterable**: able to be broken down into smaller parts of equal size that can be processed in turn. You can loop through any iterable object.
- **Slice**: a group of neighboring elements in a sequence.
- **List**: a mutable data type in Python that can store many types of data. The most common data structure in Python.
- **Tuple**: an immutable data type in Python that can store many types of data.
- **Range**: a data type in Python that stores integers in a fixed pattern.
- **String**: an immutable data type in Python that stores unicode characters in a fixed pattern. Iterable and indexed, just like other sequences.

Introduction

In this lesson, we'll build the definition for a `Set` class along with some of its common methods to get an understanding of the general approach to building data structures.

Building Data Structures From Scratch

When you're making a data structure to solve a particular algorithm problem, you won't need to implement **every** common method of that data structure: you only need to implement the ones that are required to solve your particular problem.

You may be wondering why you need to make a `Set` yourself, since many languages, including Python and JavaScript, already have a built-in `Set` class. It's common in interview settings that you'll be allowed to use built-in classes, so you won't necessarily need to be able to build a `Set` from scratch. However, building a data structure from scratch is a useful exercise for a few reasons:

- It gives you a better understanding of the Big O of common methods in built-in classes, which you'll need to know in order to determine your algorithm's efficiency.
- Many other common data structures, like Linked Lists, Stacks, and Queues, **aren't** included in some languages, so it's important to know a general process to building data structures.
- Not all languages implement data structures in the same way. For example, the `Set` in JavaScript (https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Set) preserves the order that elements were inserted into the `Set`, whereas the `Set` in python (https://docs.python.org/3/tutorial/datastructures.html#sets) does not.

When building data structures from scratch, you'll often use other built-in data structures to support your data structure and hold data. A key consideration when building on top of built-in data structures is understanding the Big O runtime of built-in methods, so make sure to keep this in mind.

When To Use a Set

A `Set` is a data structure that is used for storing a collection of **unique** values. They are useful for problems that involve finding repeated values, or removing duplicate values.

For example, here's an algorithm for finding the first repeated value in an list. Without using a `Set`, we might end up with a solution like this that has a O(n²) runtime, since we need to check every element in the list against all the remaining elements:

```
def first_repeated_value(list):
    for i in range(0, len(list)):
        for j in range(i+1, len(list)):
            if list[i] == list[j]:
                return list[i]
    return None

print(first_repeated_value([1,2,3,3,4,4]))
# => 3
```

With a `Set`, we can keep track of the values we've already seen and end up with a more efficient O(n) runtime solution, provided that the methods that allow us to access and insert have O(1) runtimes:

```
def first_repeated_value(list):
  # create a Set to keep track of values we've seen
  number_set = set()
  # iterate over each element from the list
  for i in range (0, len(list)):

    # if we've already seen a value, we've found the duplicate!
    if list[i] in number_set:
        return list[i]
    # otherwise, add the value to our set
    number_set.add(list[i])
  # return None if we reach the end and haven't found our value
  return None

first_repeated_value([1,2,3,3,4,4])
# => 3
```

Defining a Set Class

Let's make our own version of Python's `Set` class to understand how these methods might work under the hood. We'll build a `MySet` class using Python that has the following methods:

- `__init__` : Initializes a new `MySet` and adds any values from a list.
- `has(value)` : Checks if the value is already included in the `MySet` . Must have a O(1) runtime.
- `add(value)` : Adds the value to the `MySet` if it isn't already present. Must have a O(1) runtime.
- `delete(value)` : Removes the value from the `MySet` . Must have a O(1) runtime.
- `size()` : Returns the number of elements in the `MySet` .

Let's get started! We'll be coding in the `lib/MySet.py` file. You can run the tests at any point using `pytest -x` to check your work.

MySet class

To start, we'll need to define a class and set up an `__init__` method:

```
class MySet:
  def __init__(self):
    pass
```

Let's think about how we might want to use this class. We may want to initialize a new, empty set:

```
set = MySet()
# => #<MySet: {}>"
```

We might also want to pass in an existing collection of values, such as an list, and create a new set with just the unique values:

```
set = MySet([1, 2, 3, 3])
# => #<MySet: {1, 2, 3}>"
```

Let's update our `__init__` method to account for these two cases:

```
class MySet
  def __init__(self, list = [])
    pass
```

Now, we need a way to keep track of all the values that were passed in. Think about this: we want to keep track of a collection of data, and we want to be able to **access** and **add** elements to that collection with O(1) runtime.

In order to do both of these operations, we'll need to use **another** data structure to keep track of the elements in our set: a `Dictionary` ! As you may recall from earlier lessons, we discussed that a `Dictionary` data structure has (roughly) the following runtimes:

Method	Big O
Access (looking for a value with a known key)	O(1)
Search (looking for a value without a known key)	O(n)
Insertion (adding a value at a known key)	O(1)
Deletion (removing a value at a known key)	O(1)

With that in mind, we can complete our `__init__` method by creating a `Dictionary` and storing the values passed in as **keys** on the `Dictionary` :

```
def __init__(self, enumerable = []):
    self.dictionary = {}
    for value in enumerable:
        self.dictionary[value] = True
```

Run the tests now: the `MySet __init__()` tests should be passing. We can create new instances of our data structure. Fantastic!

MySet.has()

Next up: the `has()` method. This method checks if the value is already included in the set, and returns `true` if so, and `false` if not. It also must have a O(1) runtime.

Since we're using a `Dictionary` as the underlying data structure for our set, what are some ways we can check if the value is present as a key in the `Dictionary` ?

We could either use bracket notation, and check if the key is present and the value is truthy:

```
def has(self, value):

  return self.dictionary[value]
```

That approach won't work as well if the key *isn't* present, since it will return `None` instead of `False` when the value isn't in our set.

Let's use the `in` keyword, which will always return either true or false:

```
def has(self, value):
  return value in self.dictionary
```

Run the tests now again to pass the `MySet.has()` tests. Fantastic!

MySet.add()

This method needs to add a value to the set if it isn't already present, and return the updated set. It also must have a O(1) runtime.

Like the `has()` method, we'll be working with our underlying `Dictionary` data structure once more. Since adding a key to a Dictionary is an O(1) runtime operation, here's what our `add()` method should look like:

```
def add(self, value)
  self.dictionary[value] = True # Add a value as a key on the Dictionary
  return self                   # Return the updated set
```

Run the tests again to make sure your `add()` method works. Only two more left!

MySet.delete()

The `delete()` method removes a value from the set, and returns the updated set. It also must have a O(1) runtime.

Once again, we're operating on the underlying `Dictionary` data structure and can take advantage of a built-in `pop` method here. The optional second argument `None` is the return value if the value does not exist in the `Dictionary` . If we do not define the optional second argument the default is an exception. not exist instead it returns `None` :

```
def delete(self, value):
  self.dictionary.pop(value, None)
  return self
```

MySet.size()

Last one! The `size()` method simply needs to return the number of elements in the set.

```
def size(self):
  return len(self.dictionary)
```

Bonus

For an extra bonus, here are some additional methods to try implementing. There are tests for these in the `lib/testing/set_test.py` file; uncomment the **bonus methods** section in the test file to try these out.

- `MySet.clear()` : Removes **all** the items from the set, and returns the updated set.
- Print the set in a readable format using the `__str__` method.

Examples:

```
set = MySet([1,2,3])
print(set)
# => MySet: {1, 2, 3}



set.clear()
print(set)
# => MySet: {}
```

Conclusion

In this lesson, we learned about some general approaches to building a data structure from scratch by implementing a `MySet` class. In doing so, we were able to better understand the use cases for this data structure, as well as the runtime of common methods. Keep in mind that the runtime of our data structure will depend on what data structure(s) it uses under the hood.

Solution Code

```
class MySet:

    def __init__(self, enumerable = []):
        self.dictionary = {}
        for value in enumerable:
            self.dictionary[value] = True

    def has(self, value):
        return value in self.dictionary

    def __str__(self):
        set_list = []
```

```
        for key, value in self.dictionary.items():
            set_list.append(str(key))
        return f'MySet: {{{",".join(set_list)}}}'

    def add(self, value):
        self.dictionary[value] = True # Add a value as a key on the Dictionary
        return self

    def delete(self, value):
        self.dictionary.pop(value, None)
        return self

    def size(self):
        return len(self.dictionary)

    def clear(self):
        self.dictionary.clear()
```

Resources

- **Python Set (https://docs.python.org/3/tutorial/datastructures.html#sets)**
- **JavaScript Set Class (https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Set)**

This tool needs to be loaded in a new browser window

Load Set Data Structure Code Along (CodeGrade) in a new window